

GNR 638 Programming Assignment 1 Report

Design of a Custom Deep Learning Framework

Afnan Abdul Gafoor
Roll Number: 22b2505

February 16, 2026

Contents

1	Introduction	2
2	Requirement Coverage	2
3	Framework Design	2
3.1	System Architecture	2
3.2	Implemented Components	3
4	Dataset Pipeline	3
5	Model and Experiment Naming	3
5.1	CNN Architecture	3
5.2	Architecture Rationale	3
5.3	Run Name Definitions	4
6	Complexity Metrics	4
7	Quantitative Results	4
7.1	Dataset Loading Time	4
7.2	Training and Evaluation Summary	4
7.3	Runtime Statistics	5
8	Plots and Analysis	5
9	Failed Design Decision	7
10	Key Insights	8
11	Resources and Honor Code Declaration	9
	Appendix	9

1 Introduction

This report presents the implementation and evaluation of a custom deep learning framework for multiclass image classification using only the tools permitted in the assignment.

Repository (Assignment 1): <https://github.com/Afnnnan/GNR638/tree/main/Assignment1>

Core constraints satisfied:

- Python 3.12 frontend with a C++ backend.
- No use of external deep learning frameworks.
- OpenCV usage limited to image loading and basic image processing.
- Complete training/evaluation pipeline executed through the custom framework.

2 Requirement Coverage

Table 1: Assignment Requirement Coverage

Requirement	Implementation Evidence
Tensor abstraction and gradient tracking	<code>dlframework/tensor.py</code> , gradient buffers in layer modules
Backpropagation support	explicit backward implementations in <code>dlframework/layers.py</code>
Core NN operations (conv, activation, pool, FC)	C++ kernels in <code>cpp_backend/kernels.cpp</code> , wrappers in <code>dlframework/backend.py</code>
Python frontend + C++ backend integration	ctypes-based backend bridge
Image-folder dataset loading and label inference	<code>utils/data.py</code>
Resize to 32×32 , batching support	<code>utils/data.py</code> , <code>train.py</code> , <code>evaluate.py</code>
Dataset loading time in train and eval	printed in logs and stored in metadata JSON
Params/MACs/FLOPs in train and eval	<code>dlframework/metrics.py</code> , printed in both scripts
Train/eval script contract from paths only	<code>train.py</code> and <code>evaluate.py</code> CLI arguments

3 Framework Design

3.1 System Architecture

- **Python layer:** model composition, optimizer orchestration, loss computation, data pipeline, checkpointing.
- **C++ layer:** performance-critical forward/backward kernels for Conv2D, Linear, ReLU, MaxPool2D.

3.2 Implemented Components

- Tensor + gradients: `dlframework/tensor.py`
- Layers: Conv2D, ReLU, MaxPool2D, Flatten, Linear, GlobalAvgPool2D
- Loss: softmax cross entropy
- Optimizer: SGD with momentum and weight decay
- Learning rate schedule and checkpoint resume support

4 Dataset Pipeline

Given a parent directory, the loader:

1. indexes subfolders,
2. infers class labels from folder names,
3. loads PNG files,
4. converts to RGB and resizes to 32×32 ,
5. applies optional augmentation (flip + crop-pad),
6. converts to CHW float tensors and batches samples.

5 Model and Experiment Naming

5.1 CNN Architecture

All main runs use a CNN satisfying assignment constraints:

$$\begin{aligned} &\text{Conv}(3 \rightarrow C_1, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2) \\ &\text{Conv}(C_1 \rightarrow C_2, 3 \times 3) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2) \\ &\text{Flatten} \rightarrow \text{FC}(\text{hidden}) \rightarrow \text{ReLU} \rightarrow \text{FC}(\text{num_classes}) \end{aligned}$$

5.2 Architecture Rationale

The selected CNN design was retained as the primary submission model for the following reasons:

- It satisfies all required layer constraints (convolution, activation, pooling, fully connected) while remaining simple and stable to optimize in a custom framework setting.
- It provides a favorable runtime-accuracy tradeoff with the available C++ kernel coverage and Python orchestration overhead.
- Empirically, tuning optimization and preprocessing (momentum, LR schedule, augmentation, normalization) produced larger gains than switching to deeper alternatives under the assignment time budget.
- The model was easy to benchmark and compare across multiple controlled runs, supporting clear analysis and reproducibility.

5.3 Run Name Definitions

- **Data1 baseline:** initial 6-epoch CNN run.
- **Data1 best fresh tuned:** new 20-epoch run with momentum, LR scheduling, augmentation and normalization.
- **Data2 baseline/continued:** earlier resumed configuration with weaker normalization and lower final performance.
- **Data2 best fresh tuned:** selected final Data2 run (45 epochs) with tuned optimization and preprocessing.

6 Complexity Metrics

Per-sample complexity:

Table 2: Model Complexity

Configuration	Parameters	MACs	FLOPs
Data1 CNN (8/16/64)	67,642	582,272	1,164,544
Data2 CNN (12/24/128)	212,588	1,204,736	2,409,472

7 Quantitative Results

7.1 Dataset Loading Time

Table 3: Measured Dataset Timing (seconds)

Run	Indexing Time	Decode/Load Time
Data1 baseline train	0.0641	5.6412
Data1 baseline eval	0.1563	7.4034
Data1 best fresh tuned train	0.1176	8.8873
Data1 best fresh tuned eval	0.1491	5.3542
Data2 baseline train	0.1053	5.4854
Data2 baseline eval	0.1922	11.6050
Data2 best fresh tuned train	0.1496	8.4663
Data2 best fresh tuned eval	0.1541	7.9385

7.2 Training and Evaluation Summary

Table 4: Main Experimental Outcomes

Experiment	Epochs	Best Val Acc	Best Val Epoch	Eval Acc	Eval Loss
Data1 baseline CNN	6	0.8413	6	0.8460	0.4629
Data1 best fresh tuned CNN	20	0.9638	14/17	0.9671	0.1027
Data2 baseline/continued CNN	21	0.2500	20	0.2624	3.0841
Data2 best fresh tuned CNN	45	0.4832	42	0.6194	1.3852

7.3 Runtime Statistics

Table 5: Epoch-Time Statistics

Run	Mean Epoch Time (s)	Min (s)	Max (s)
Data1 baseline (6 epochs)	146.44	143.67	150.29
Data1 best fresh tuned (20 epochs)	160.70	154.28	172.77
Data2 baseline/continued (21 epochs)	241.78	231.30	283.04
Data2 best fresh tuned (45 epochs)	239.05	230.80	249.72

8 Plots and Analysis

Figure 1b and Figure 2b show the shift from the initial Data1 baseline to the tuned run, with a clear uplift in validation performance. Figure 3b and Figure 4a provide the same baseline-versus-best view for Dataset 2. Figure 5 summarizes evaluation accuracy across major runs.

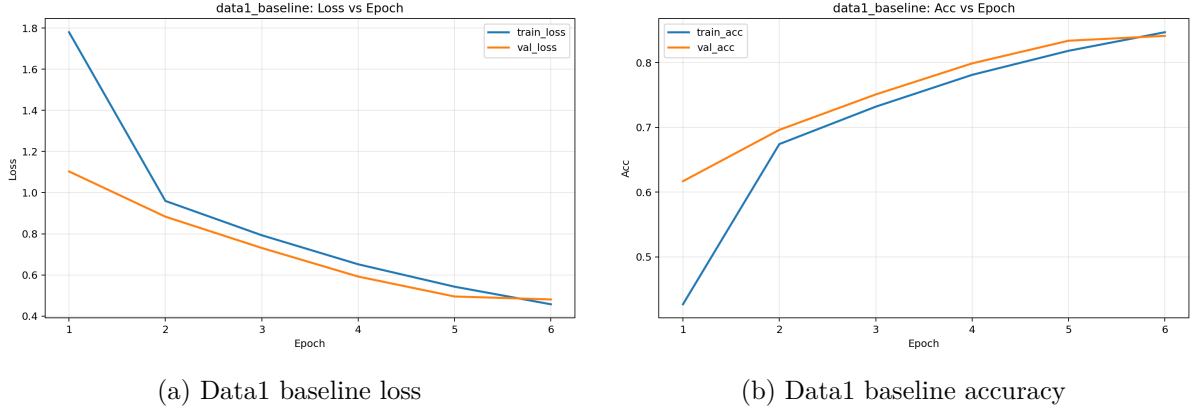


Figure 1: Data1 baseline run

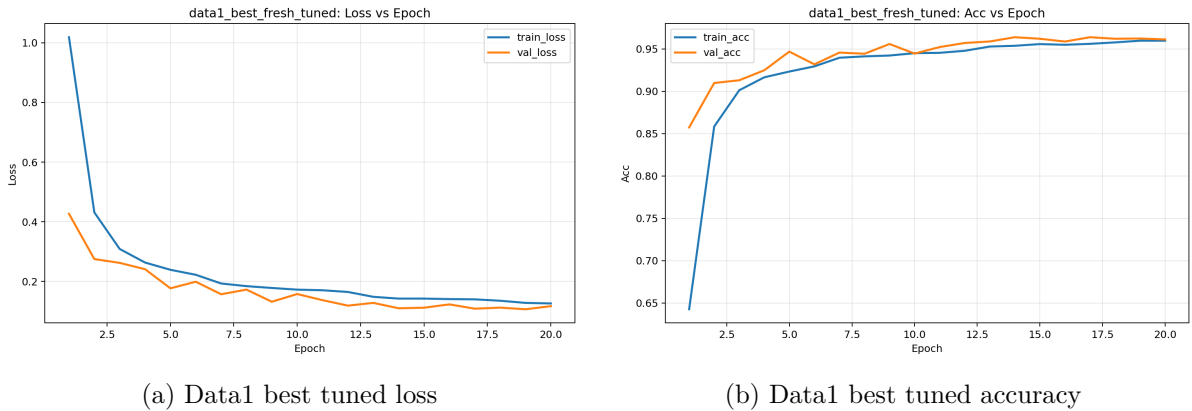
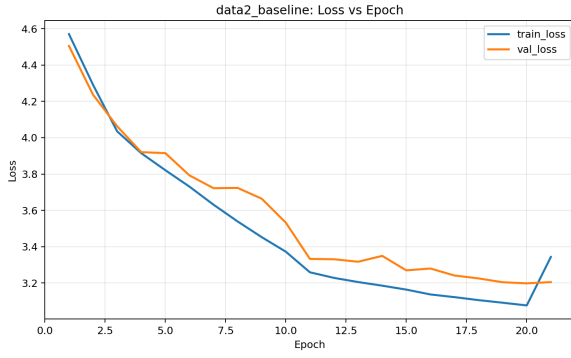
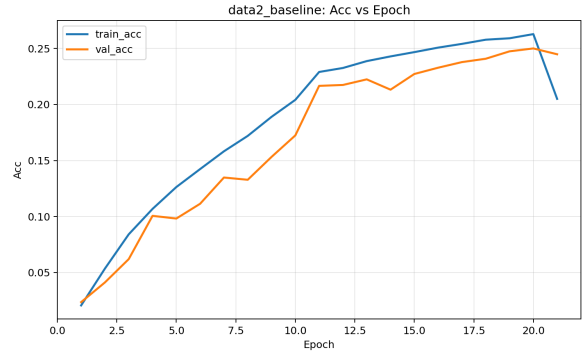


Figure 2: **Data1 best fresh tuned run (latest)**

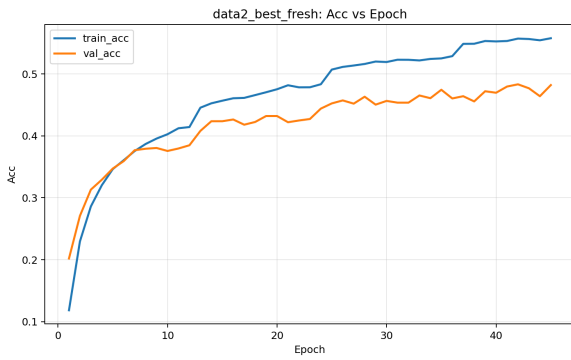


(a) Data2 baseline loss



(b) Data2 baseline accuracy

Figure 3: Data2 baseline run



(a) Data2 best tuned accuracy



(b) Data2 best tuned loss

Figure 4: Data2 best fresh tuned run

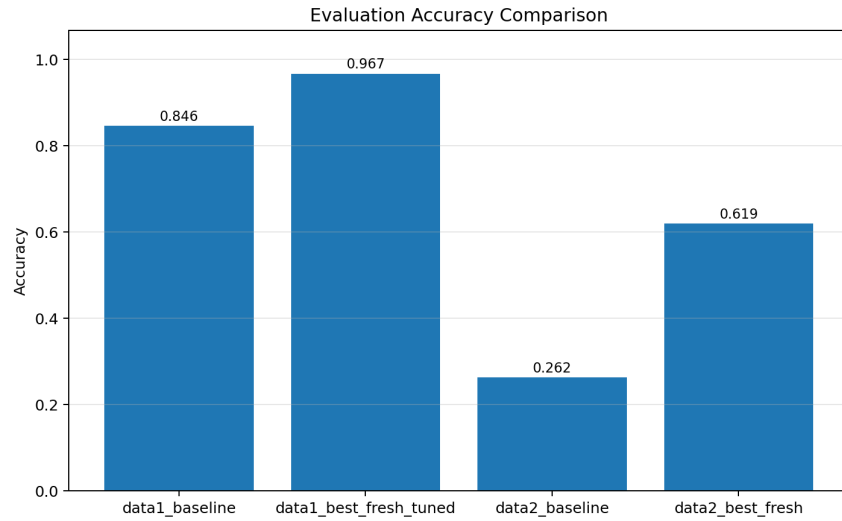


Figure 5: Evaluation accuracy comparison across major runs

9 Failed Design Decision

The key failed design decision was starting with a Python-first backend for core tensor kernels (nested-list loops for convolution, pooling, matrix multiplication, and backward passes). The approach was functionally correct but failed runtime expectations for iterative experimentation.

Phase 1: Pure Python Backend

Measured on Dataset 1:

- Dataset load time: ~ 62.6 s
- Per-batch timing (batch size = 32):
 - Forward: ~ 0.70 s
 - Backward: ~ 0.79 s
 - Optimizer step: ~ 0.01 s
 - Total: ~ 1.52 s/batch

Epoch-time estimate:

$$\text{batches/epoch} = \frac{60000}{32} = 1875, \quad \text{epoch time} \approx 1875 \times 1.52 = 2849 \text{ s} \approx 47.5 \text{ min}$$

$$5 \text{ epochs} \approx 14245 \text{ s} \approx 3.95 \text{ h (excluding extra overhead)}$$

Root cause:

- Heavy kernels dominated by Python-level nested loops.
- No vectorized numerical backend in pure Python path.
- Full preload data strategy (nested Python objects/lists) added memory and interpreter overhead.

Phase 2: Hybrid Python + C++ Backend

Observed results on Dataset 1:

- Eval load time: 35.8139 s
- Parameters: 18,234
- MACs/FLOPs: 212,400 / 424,800
- Eval accuracy: 0.8773
- Training epoch 1: loss=1.2342, acc=0.6107, time=981.53 s

Interpretation: hybrid execution improved runtime substantially relative to pure-Python estimates (~ 2849 s $\rightarrow \sim 981.53$ s per epoch), but remained too slow for broad hyperparameter exploration.

Phase 3: OpenMP Parallelization Attempt

Observed results on Dataset 1:

- Train load time: 35.5084 s
- Parameters: 18,234
- MACs/FLOPs: 212,400 / 424,800
- Epoch 1: loss=1.1469, acc=0.6479, time=814.83 s

Speedup over hybrid baseline:

$$\Delta t = 981.53 - 814.83 = 166.70 \text{ s}, \quad \text{relative gain} \approx 16.98\% (\sim 17.0\%)$$

Interpretation: OpenMP delivered incremental gains but not transformative end-to-end improvement, because Python-side orchestration and data movement remained major contributors.

Conclusion from Failure

This failure sequence directly motivated the final redesign: prioritize a compute-centric C++ kernel path with reduced Python overhead in the critical training loop. That redesign is reflected in the final successful runs summarized in Table 4.

10 Key Insights

- **Optimization dynamics dominated final performance:** on Dataset 2, moving from the earlier baseline/continued setup (Eval Acc = 0.2624) to the tuned fresh run (Eval Acc = 0.6194) produced a large gain without changing the core CNN topology.
- **Preprocessing quality directly impacted generalization:** the tuned configuration with crop-pad augmentation and CIFAR-style normalization achieved substantially better validation behavior than weaker normalization settings.
- **Runtime bottleneck insight:** the progression from pure Python kernels → hybrid C++ kernels → OpenMP showed that kernel acceleration helps, but end-to-end training speed still depends on Python orchestration and data movement overhead.
- **Capacity vs efficiency trade-off:** in this framework context, a well-tuned shallow CNN delivered better practical accuracy-time tradeoff than the attempted tiny-ResNet route.
- **Learning-curve insight:** Dataset 2 exhibited clear early underfitting; longer training with scheduled LR decay converted steady epoch-wise improvements into strong final evaluation accuracy.
- **Final selected runs for submission are Data1 best fresh tuned and Data2 best fresh tuned.**

11 Resources and Honor Code Declaration

Resources used during this assignment:

- Course lectures and course-provided assignment material.
- Open-source resources and documentation used for conceptual understanding and debugging.
- AI tools used for assistance in code development and report writing.

All implementation and reporting decisions were carried out in accordance with course honor-code expectations.

Appendix

A. Data Split, Preprocessing, and Batching Details

Train/Validation Split

The split behavior implemented in `utils/data.py` is:

$$\text{val_size} = \lfloor N \cdot \text{val_ratio} \rfloor$$

with deterministic shuffling based on `seed`. Validation samples are the first `val_size` items after shuffle; remaining samples form training data.

Preprocessing Pipeline Order

For each image:

1. Read with OpenCV in unchanged mode.
2. Convert to RGB:
 - grayscale $\rightarrow$ RGB,
 - BGRA $\rightarrow$ RGB,
 - BGR $\rightarrow$ RGB.
3. Resize to 32×32 if needed.
4. Optional train-time augmentation:
 - reflection padding by `augment_crop_pad`,
 - random crop back to 32×32 ,
 - random horizontal flip with probability 0.5.
5. Convert to CHW float tensor and apply channel normalization:

$$x' = \frac{x/255 - \mu}{\sigma}$$

Evaluation disables augmentation and uses the same normalization parameters as training.

B. Configuration Semantics

Table 6 summarizes the main config keys used by `train.py/evaluate.py`.

Table 6: Important Configuration Keys

Key	Type	Meaning
<code>seed</code>	int	Random seed for split/shuffle/augmentation sampling
<code>epochs</code>	int	Target epoch count (absolute count when resuming)
<code>batch_size</code>	int	Training/eval minibatch size
<code>lr</code>	float	Base learning rate
<code>lr_decay_every</code>	int	Step interval (epochs) for LR decay; 0 disables
<code>lr_decay_gamma</code>	float	Multiplicative decay factor
<code>momentum</code>	float	SGD momentum coefficient
<code>weight_decay</code>	float	L2-style weight decay term in optimizer step
<code>val_ratio</code>	float	Fraction for validation split from train folder
<code>augment</code>	bool	Enable train-time data augmentation
<code>augment_crop_pad</code>	int	Reflection-pad pixels used before random crop
<code>normalize_mean/std</code>	list[float]	Channel-wise normalization parameters
<code>model_type</code>	str	<code>cnn</code> or <code>resnet</code>
<code>conv1_out/conv2_out/hidden</code>	int	CNN width and MLP head size
<code>resnet_base_channels</code>	int	Base width for tiny ResNet variant
<code>out_dir</code>	str	Directory for logs and checkpoints
<code>resume_path</code>	str	Optional checkpoint path for resume
<code>max_train_samples/ max_val_samples</code>	int	Optional truncation for quick experiments

C. Learning Rate and Resume Logic

LR Schedule

If `lr_decay_every` > 0, the effective LR at epoch e is:

$$\eta_e = \eta_0 \cdot (\gamma)^{\lfloor \frac{e-1}{\text{lr_decay_every}} \rfloor}$$

where η_0 is `lr` and γ is `lr_decay_gamma`.

Resume Behavior

When `resume_path` is provided:

- model parameters are loaded from checkpoint,
- `last_epoch`, `best_val_acc`, `best_val_loss` are restored from metadata,
- existing `train_log.json` (if present) is merged to continue history,
- training resumes from `(start_epoch + 1)`.

D. Checkpoint and Log Schema

Checkpoint File

Checkpoint JSON contains:

- **metadata:** config + complexity + best metrics + class names
- **params:** flattened parameter arrays
- **shapes:** tensor shapes per parameter

Training Log File

`train_log.json` stores:

- **history:** epoch-wise train/val loss, accuracy, LR, epoch time
- **metadata:** run metadata snapshot

E. Complexity Calculation Notes

Complexity is reported per single-sample forward pass. For CNN:

$$\text{MACs} = \text{MACs}_{\text{conv1}} + \text{MACs}_{\text{conv2}} + \text{MACs}_{\text{fc1}} + \text{MACs}_{\text{fc2}}$$

and FLOPs are approximated as:

$$\text{FLOPs} \approx 2 \times \text{MACs}$$

Trainable parameters include all convolution and linear weights plus biases.

F. Runtime Artifacts Produced

For each run directory `outputs/<run_name>/`:

- `model_checkpoint.json`: latest checkpoint
- `model_checkpoint_best.json`: best validation checkpoint
- `train_log.json`: epoch-wise metrics
- `eval_metrics.json`: evaluation summary (after eval script)

Execution Environment

All reported experiments were run on a 2021 MacBook Pro (Apple M1 Pro, 8-core CPU).